

Aula 3

Inteligência Artificial Aplicada - Machine Learning

Prof. Antonio Willian Sousa

1

Conversa Inicial

2

- ▀ **Preparação dos dados**
- ▀ **Árvores de decisão**
- ▀ **Treinamento do modelo**
- ▀ **Usando modelos treinados**
- ▀ **Outros algoritmos de árvore**

3

Preparação dos dados

4

Obtenção dos dados

- ▀ **Quantidade x qualidade**
- ▀ **Relevância dos dados**
- ▀ **Conversão e agregação de dados**
- ▀ **Determinação dos atributos relevantes**
- ▀ **Possível necessidade de conversões**

5

Considerações sobre os dados

- ▀ **Localização em banco de dados**
- ▀ **Relação temporal**
- ▀ **Disponibilidade**
- ▀ **Produção de dados sintéticos**

6

Extração de características

- Agregação de valores
 - Ex.: quantidade de ligações, de reservas
- Decomposição de dados
 - Ex.: 10/09/2019 20:30h ou
terça-feira, 10, 09, 2019, 20:30h
- Composição
 - Ex.: data de check in – data de reserva

7

- Normalização de atributos por escala
 - Geralmente coloca-se entre [0, 1]
- Normalização por padronização
 - Considera a distribuição dos dados
 - Cada atributo terá médio 0 e desvio-padrão 1
- A operação tem que ser reversível

8

Separação, treino e testes

- 80-20 é um bom começo
 - É preciso ter variabilidade
- Validação cruzada
 - K-1 para treino, 1 para testes
 - K-1 etapas

9

Separação treino e testes

```
from sklearn.model_selection import train_test_split
import pandas as pd
# Lê o dataset de carros
carros = pd.read_csv("carros.csv")
X = carros.loc[:, 0:-1].to_numpy()
y = carros['label'].to_numpy()
# Divide o conjunto em treinamento e teste, na
# proporção 80-20
X_treino, X_teste, y_treino, y_teste =
train_test_split(X, y, test_size=0.20)
```

10

```
from sklearn.model_selection import Kfold
import pandas as pd

# Lê o dataset de carros
carros = pd.read_csv("carros.csv")
X = carros.loc[:, 0:-1].to_numpy()
y = carros['label'].to_numpy()

# Divide o conjunto em 3 folds
kf = KFold(n_splits=3)
idx_folds = kf.split(X)
for idx_treino, idx_teste in idx_folds:
    X_treino, X_teste = X[idx_treino], X[idx_teste]
    y_treino, y_teste = y[idx_treino], y[idx_teste]
```

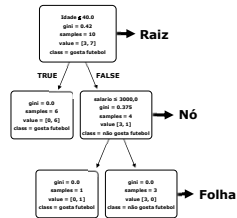
11

Árvores de decisão

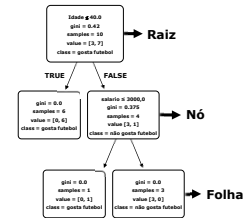
12

Como funciona

	NOME	IDADE	NACIONALIDADE	ALTURA	PESO	SEXO	GOSTA DE FUTEBOL	SALARIO
0	Adão	22	0	1,70	75	0	1	2.500
1	Beto	52	1	1,75	80	0	1	3.500
2	Bia	31	1	1,80	65	1	1	4.500
3	Carlos	45	1	1,85	85	0	0	6.500
4	Diego	17	0	1,85	90	0	1	1.500
5	Esther	34	0	1,85	80	1	0	3.500
6	Lucas	30	0	1,90	100	1	1	1.500
7	Fabio	25	1	1,85	60	1	1	2.100
8	Fabio	48	0	1,71	75	0	0	10.000
9	Guia	26	1	1,81	80	1	1	6.000



- Atributos são utilizados na separação
- Samples: quantidade de amostras daquele nó
- Gini: métrica de pureza do nó
- Value: lista de amostras que atendem ao critério
- Class: categoria das amostras



Tipos de algoritmos

- Iterative Dichotomiser 3 (ID3)
- C4.5
- C5.0
- Classification and Regression Trees (CART)

Treinamento do modelo

Dataset real

ano	combustivel	tp	cilindros	tipc	transmissao	tracao	tracao_4p	categoria	tanque	motor	vel_max	vel_max_litros	populacao	preco	label
2011	gasolina	premium	150.0	6.0	MANUAL	tracao_4p	2.0	Alfa Romeo, Alfa-performance	compacto	Conversivel	26	19	2016	46135	BMW 1 Series M
2011	gasolina	premium	200.0	6.0	MANUAL	tracao_4p	2.0	Lotus, Alfa-performance	compacto	Conversivel	26	19	2016	48659	BMW 1 Series
2011	gasolina	premium	300.0	6.0	MANUAL	tracao_4p	2.0	Lotus, Alfa-performance	compacto	Cupé	26	20	2016	36358	BMW 1 Series
2011	gasolina	premium	150.0	6.0	MANUAL	tracao_4p	2.0	Lotus, Alfa-performance	compacto	Cupé	26	18	2016	26450	BMW 1 Series
2011	gasolina	premium	150.0	6.0	MANUAL	tracao_4p	2.0	Lotus	compacto	Conversivel	26	16	2016	24506	BMW 1 Series
2011	gasolina	premium	150.0	6.0	MANUAL	tracao_4p	2.0	Lotus, Alfa-performance	compacto	Cupé	26	18	2016	31280	BMW 1 Series
2012	gasolina	premium	200.0	6.0	MANUAL	tracao_4p	2.0	Lotus, Alfa-performance	compacto	Conversivel	26	17	2016	44100	BMW 1 Series
2012	gasolina	premium	200.0	6.0	MANUAL	tracao_4p	2.0	Lotus, Alfa-performance	compacto	Cupé	26	20	2016	39395	BMW 1 Series
2012	gasolina	premium	150.0	6.0	MANUAL	tracao_4p	2.0	Lotus	compacto	Conversivel	26	18	2016	36000	BMW 1 Series
2013	gasolina	premium	150.0	6.0	MANUAL	tracao_4p	2.0	Lotus	compacto	Conversivel	27	18	2016	37280	BMW 1 Series

- O treinamento deve ter amostras de todas as categorias
- O teste também
- Características devem ser convertidas em números
- Classes ou labels são os modelos de automóveis

DATASET DE AUTOMÓVEIS	
Total de registros do dataset	11913
Tamanho do conjunto de treino	9427
Tamanho do conjunto de testes	2357
Quantidade de características	14
Quantidade de classes	48

Treinando árvore de decisão

```
import pandas as pd
from sklearn import tree
from sklearn.model_selection import train_test_split

# Lê o dataset de carros
carros = pd.read_csv("carros_categorical.csv")
```

19

```
# Seleciona apenas as colunas necessárias
X = carros.loc[:, 'ano':'preco_venda'].to_numpy()
y = carros['label'].to_numpy()

# Divide o conjunto em treinamento e teste, na
proporção 80-20
X_treino, X_teste, y_treino, y_teste =
train_test_split(X, y,
                 test_size=0.20,
                 random_state=42)
```

20

- **criterion:** define a medida de impureza dos nós
- **splitter:** define como as divisões serão feitas
- **max_depth:** profundidade máxima da árvore
- **min_samples_split:** número mínimo de amostras de um nó
- **min_samples_leaf:** número mínimo de amostras de uma folha
- **max_features:** número de atributos para definir a divisão

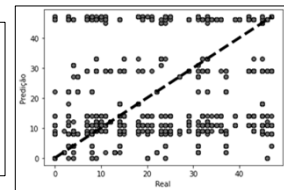
```
# Executa a definição e treinamento do modelo
clf = tree.DecisionTreeClassifier(criterion = "gini",
                                splitter = "best",
                                max_depth = 5,
                                min_samples_split = 5,
                                min_samples_leaf = 2,
                                max_features = 2)
clf = clf.fit(X_treino, y_treino)
```

21

Usando o modelo treinado

```
# Executa a definição e treinamento do modelo
clf = tree.DecisionTreeClassifier(criterion = "gini",
                                splitter = "best",
                                max_depth = 5,
                                min_samples_split = 5,
                                min_samples_leaf = 2,
                                max_features = 2)
clf = clf.fit(X_treino, y_treino)

# Predição do conjunto de testes
y_pred = clf.predict(X_teste)
```



22

Avaliando o desempenho

		VALORES REAIS	
		V	F
PREDIÇÃO DO MODELO	V	Verdadeiros Positivos	Falsos Positivos
	F	Falsos Negativos	Verdadeiros Negativos

DEFINIÇÃO	SIGNIFICADO
Verdadeiros Positivos	Instâncias preditas como positivas pelo modelo, que realmente são positivas
Verdadeiros Negativos	Instâncias preditas como negativas pelo modelo, que realmente são negativas
Falsos Positivos (ou erro tipo I)	Instâncias preditas como positivas pelo modelo, que na verdade são negativas
Falsos Negativos (ou erro tipo II)	Instâncias preditas como negativas pelo modelo, que na verdade são realmente positivas

23

- Indica quantas predições foram feitas corretamente
- Fácil de analisar
- Não deve ser considerada de forma isolada

		REAL	
		CÂNCER	NÃO CÂNCER
MODELO	CÂNCER	1	1
	NÃO CÂNCER	0	98

$$\text{Acurácia} = \frac{\text{verdadeiros positivos} + \text{verdadeiros negativos}}{\text{total de instâncias}}$$

24

- Indica, do total de predições, quantas de fato estão corretas
- Precisão baixa = muitos falsos positivos
- Custo dos falsos positivos é alto
 - Ex.: detecção de spam

Precisão =	$\frac{\text{verdadeiros positivos}}{\text{verdadeiros positivos} + \text{falsos positivos}}$
------------	---

25

- Indica, do total de positivos, quantos foram preditos corretamente
- Recall baixo = muitos falsos negativos
- Custo dos falsos negativos é alto
 - Ex.: detecção de fraude

Recall =	$\frac{\text{verdadeiros positivos}}{\text{verdadeiros positivos} + \text{falsos negativos}}$
----------	---

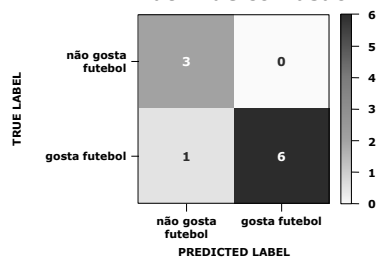
26

- Diferente de precisão
- Melhor para um equilíbrio entre precisão e recall
- Nenhuma métrica isolada deve ser suficiente

F1 = 2 x	$\frac{\text{precisão} \times \text{recall}}{\text{precisão} + \text{recall}}$
----------	--

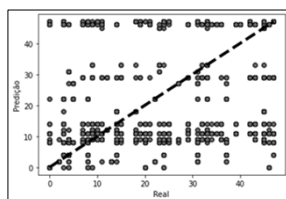
27

Matriz de confusão



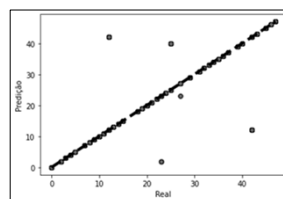
28

Melhorando um modelo



- max_depth = 5
- max_features = 2

29



- max_depth = 15
- max_features = 14

30

Utilizando um modelo treinado

31

Serialização

- ▀ Converte objetos da linguagem em sequência de bits
- ▀ Comum em diversas linguagens
- ▀ Disponível no Python
- ▀ Biblioteca Pickle

32

Serialização/deserialização

```
import pickle

with open("clf_automoveis_dt.pickle", 'wb') as f:
    pickle.dump(clf, f, pickle.HIGHEST_PROTOCOL)

import pickle
from sklearn import tree

clf = None
with open("clf_automoveis_dt.pickle", 'rb') as f:
    clf = pickle.load(f)
```

33

Boas práticas de serialização

- ▀ Disponibilidade dos dados de treino
- ▀ Código da geração do modelo
- ▀ Versão das bibliotecas utilizadas
- ▀ Valores obtidos no treinamento
- ▀ Versão da biblioteca de serialização

34

Outras formas de serialização

- ▀ Biblioteca Joblib
- ▀ Utilizar JSON
- ▀ Resultado da serialização pode ser alterado
- ▀ Risco de segurança

35

Combinação de modelos

36

Ensembles

- Metaclassificadores (ou regressores)
- Combinam diferentes modelos
- Podem combinar diferentes algoritmos
- Podem melhorar predição
- Podem reduzir bias
- Podem reduzir variância

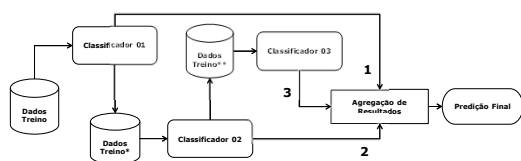
37

- Reduzir variância = reduzir overfitting
- Reduzir bias = reduzir underfitting
- Podem ser classificados em
 - Sequenciais: exploram dependência
 - Paralelos: exploram independência

38

Ensembles sequenciais

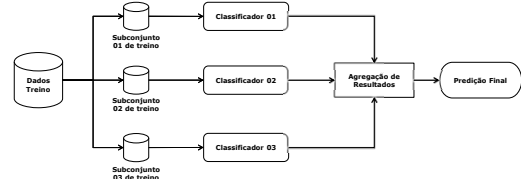
- Abordagem mais comum boosting. Exemplo: AdaBoost



39

Ensembles paralelos

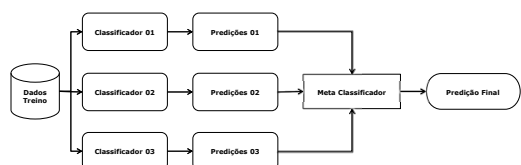
- Abordagem mais comum bagging. Exemplo: RandomForest



40

Ensembles stacking

- Utiliza diferentes algoritmos



41

Outros ensembles de árvores

- Isolation Forest
- Extra Trees Classifier
- AdaBoost com Árvore de Decisão

42